Cost-Sensitive Machine Learning for Accounting Transaction Review

MSc Corporate Financial Management
Project brief

1 Business challenge

You work as an analyst in a corporate finance department. The accounting team processes transactions automatically, but some transactions fail. Reviewing transactions takes time and resources, while leaving failures undetected can create additional work and operational losses.

Your task is to design and evaluate a policy that identifies which transactions should receive human review **before automated processing**.

> **Which combination of predictive model and decision threshold minimises the cost of missed failures and unnecessary alerts?**

You will prepare features in Excel and use an instructor-provided notebook to train and evaluate models. **No Python programming is required.** You are responsible for your assumptions, feature choices, interpretation and recommendation.

2 Learning objectives

By completing this project, you should be able to:

1. Translate an accounting problem into a supervised classification task.
2. Create and justify features using Excel.
3. Distinguish training, validation and final test data.
4. Identify information leakage and exclude unavailable information.
5. Interpret a confusion matrix, precision and recall.
6. Select a model and a decision threshold using explicit error costs.
7. Communicate a recommendation and its operational limitations.

3 Dataset and prediction target

The proposed dataset contains 1,000 simulated accounting transaction records. Its description includes transaction identifiers, dates, account types, amounts, financial indicators, processing measures and an outcome.

The original target is *Transaction Outcome*, coded as success = 1 and failure = 0. The notebook will define:

$$y_i = 1 - \text{Transaction Outcome}_i.$$

Thus, $y_i = 1$ represents a **processing failure**, the positive class. The model produces a failure-risk estimate $\hat{p}_i$. A transaction is flagged when:

$$\hat{y}_i = \begin{cases} 1 & \text{if } \hat{p}_i \geq t, \\ 0 & \text{otherwise,} \end{cases}$$

where t is the decision threshold. A flag is a recommendation for review; it does not establish fraud or an accounting misstatement.

Data limitation. These are simulated observations. Predictive relationships may be weak or absent. A well-supported recommendation not to deploy a model is an acceptable outcome. Do not alter outcome labels or invent relationships to improve results.

4 Part 1 — Audit the data and define eligible features

Use the development data supplied by the instructor to:

1. Check row counts, missing values, duplicates, data types and date formats.
2. Calculate the proportion of failures and describe any class imbalance.
3. Examine transaction amounts and account-type distributions.
4. Identify implausible values and document how you handle them.
5. Decide whether each candidate variable would be known before processing.

Variable or group	**Required treatment**
Transaction ID	Keep as a record key; exclude from predictors.
Date, Account Type, Transaction Amount	Candidate inputs, subject to data quality and availability checks.
Transaction Outcome	Target only. Never include it, or a transformation of it, among predictors.
Processing Time, Accuracy Score	Exclude: these appear to describe processing and may reveal the outcome.
Normalized Transaction Amount	Exclude the supplied version. Any required scaling is fitted in the notebook using training data only.
Missing Data Indicator	Use only if its definition is known and it can be computed before processing.
Other financial indicators	Use only after the instructor confirms their meaning, aggregation level and pre-processing availability.

For example, a debt-to-equity ratio usually describes a company at a point in time. Its meaning on a transaction row must be established. Unclear variables should be excluded until clarified.

5 Part 2 — Engineer features in Excel

Keep an unchanged `Raw_Data` sheet and prepare a separate `Features` sheet. Retain one row per transaction, the record identifier and the original target. Do not remove difficult observations without a documented data-quality reason.

Create **three to five additional features**. Possible examples are:

Feature	Excel transformation	Hypothesis to investigate
Absolute amount	Absolute value of the transaction amount.	Transaction size may relate to processing difficulty.
Zero-amount flag	Logical test for a zero amount.	Zero amounts may represent special processing cases.
Negative-amount flag	Logical test for a negative amount.	Reversals or credits may follow different processes.
Calendar feature	Extract month or weekday from the date.	Failure rates may vary across periods.
Missing-input count	Count missing cells across a fixed list of eligible input columns.	Incomplete inputs may make processing harder.

These are hypotheses, not established properties of the dataset. You may propose alternatives if they are available at prediction time.

For each created feature, document its name, source columns, Excel formula, treatment of missing values and business rationale. Exclude the target and post-processing fields from missing-input counts. Preserve blanks in numerical source fields rather than silently converting them to zero.

Preparation boundary. Perform deterministic, row-level transformations in Excel. Leave mean/median imputation, category encoding and standardisation to the notebook. Do not calculate these transformations from the full dataset. If you create business cut-offs, define them in advance and justify them rather than selecting them from test outcomes.

Export the feature sheet as a UTF-8 CSV with one header row, unique column names, consistent dates and a documented decimal separator.

6 Part 3 — Use the ready-to-run notebook

6.1 What you control

The instructor will supply the executable notebook separately. You will load your CSV, adjust the clearly identified configuration fields and run the notebook in order. You will not be asked to write or modify modelling code.

Setting	Available choice
Input data	Your Excel-prepared CSV file.
Predictor columns	An approved subset of original and engineered features.
Training proportion	60%, 70% or 80% of the development data; the remainder is used for validation.
Model	Logistic regression or a decision tree.
Tree depth	2, 4 or 6; other settings remain fixed.
Decision threshold	A supplied grid, for example 0.05 to 0.95 in steps of 0.05, including 0.50.
Error costs	The three compulsory scenarios below; an additional justified scenario is optional.

6.2 Data splitting and fair comparison

The instructor reserves a fixed **20% final test set**. The other 80% forms the development set. With 1,000 usable records, this means 200 test records and 800 development records. Selecting 70% for training then gives 560 training and 240 validation records.

Use a fixed split seed supplied by the instructor and never search for a favourable seed. The instructor will specify stratified splits, or chronological splits if the dates support a future-transaction prediction task. All models within an experiment must use the same split.

The notebook fits preprocessing and models on training data only. Validation data supports feature, model, complexity and threshold selection. To keep comparisons simple, use **70% as the main training proportion**, and investigate 60% and 80% as a limited sensitivity analysis. Explain that these settings change both training size and validation composition.

Lock your feature set, training proportion, model and threshold for each cost scenario before the final test. The instructor applies the locked Excel feature recipes to the held-out records. Evaluate the selected fitted pipelines without retuning or refitting them, so that the tested models retain the thresholds selected on validation.

Do not use the final test set to select features, models or thresholds. If its results are disappointing, report this finding rather than selecting a different solution after seeing the test results.

6.3 Required experiments

1. Build a reference feature set from approved original variables.
2. Build an enriched feature set that adds your Excel-engineered features.
3. Compare logistic regression and decision trees using both sets.
4. For each cost scenario, compare thresholds on validation data.
5. Examine the effect of training size for one promising configuration.
6. Select one complete policy per scenario and justify the choice.

Keep a concise experiment log. It should record the feature set, split setting, model, tree depth if relevant, threshold, validation results and the reason for each change.

7 Part 4 — Evaluate mistakes in financial terms

7.1 Confusion matrix

	Flag for review	**Do not flag**
Actual failure	True positive (TP): a failure is identified.	False negative (FN): a failure is missed.
Actual success	False positive (FP): an unnecessary alert.	True negative (TN): a successful transaction is not flagged.

Report accuracy, precision, recall, F1-score and review rate:

$$\text{Accuracy} = \frac{TP+TN}{N}, \qquad \text{Precision} = \frac{TP}{TP+FP},$$
$$\text{Recall} = \frac{TP}{TP+FN}, \qquad F_1 = \frac{2TP}{2TP+FP+FN},$$
$$\text{Review rate} = \frac{TP+FP}{N}, \qquad N = TP+FP+FN+TN.$$

When a denominator is zero, the notebook should identify the metric as undefined and explain any display convention. Accuracy alone is not a suitable basis for selection when failures are uncommon or error costs differ.

7.2 Compulsory cost scenarios

Let c_{FN} be the cost assigned to one missed failure and c_{FP} the cost assigned to one unnecessary alert. Use:

$$C_{\text{errors}} = c_{\text{FN}}FN + c_{\text{FP}}FP, \qquad C_{1000} = \frac{1000}{N}C_{\text{errors}}.$$

Scenario	c_{FN} **(EUR)**	c_{FP} **(EUR)**
A — Symmetric error costs	20	20
B — Expensive missed failures	200	10
C — Expensive unnecessary alerts	50	40

These amounts are fictional teaching assumptions. This is a **simplified error-cost score**, not a full operating budget: it assigns zero cost to correct classifications and does not price all reviews or residual failures.

For each scenario, select the model–threshold combination with the lowest validation cost. If costs tie, prefer the lower review rate, then the simpler policy. Report differences that are small rather than treating every numerical improvement as meaningful.

7.3 Required baseline policies

Compare each selected policy against:

- **Flag none:** every failure is missed; $C = c_{\text{FN}}n_{\text{failure}}$.
- **Flag all:** every successful transaction creates an unnecessary alert; $C = c_{\text{FP}}n_{\text{success}}$.
- **Majority-class classifier:** always predict the most frequent training class. Explain why its decisions coincide with one of the two policies above.

The notebook should produce a comparison table with TP, FP, FN, TN, precision, recall, F1-score, review rate and cost per 1,000 transactions. Include a validation cost-versus-threshold plot for each scenario and a confusion matrix for each selected policy.

Interpretation questions

1. Does the preferred policy change when missed failures become more costly?
2. Does changing the threshold matter more than changing the model?
3. Do your engineered features improve cost, and is the improvement consistent?

4. Does ML outperform the best simple baseline?
5. How many reviews would the proposed policy generate per 1,000 transactions?
6. What uncertainty remains given the small, simulated dataset?

8 Optional extension — Review capacity and operating cost

Suppose the team can review at most 10% of incoming transactions. Rank transactions by estimated failure risk and select the top 10%, using a fixed tie-breaking rule. Report the proportion of actual failures captured. Compare this with ranking by absolute transaction amount and with the expected performance of random selection.

This ranking policy is distinct from a fixed probability threshold: its effective threshold varies with the incoming batch.

For a fuller economic analysis, define a review cost c_R, a failure loss c_F, and an assumed prevention rate $q \in [0, 1]$. If each flagged transaction is reviewed, review prevents a fraction q of flagged failures, and prevented failures avoid the full loss:

$$C_{\text{operating}} = c_R(TP + FP) + c_F\big[FN + (1 - q)TP\big].$$

This includes reviews of both true and false positives. State all assumptions, including that review does not create additional failures. Test several values of q. The dataset does not establish review effectiveness, so any estimated benefit is conditional on these assumptions.

9 Deliverables

1. **Excel workbook and CSV:** unchanged raw development data, engineered features, and a feature dictionary documenting formulas and rationale.
2. **Executed notebook:** configuration values, validation experiments, selected policies and final test outputs. No additional programming is required.
3. **Experiment log:** a compact table showing the configurations tested and the basis for selection.
4. **Two-page management memo in English:** business objective, recommended policy for each scenario, comparison with baselines, expected review workload, limitations and a deployment or non-deployment recommendation. Put detailed tables in an appendix.

All results must be traceable to submitted data and settings. Team size and deadline will be communicated by the instructor.

10 Assessment criteria

Criterion	**Weight**
Business framing, data audit and feature eligibility	20%
Excel feature engineering, documentation and reproducibility	20%
Experimental design and correct use of training/validation/test data	20%
Cost-based selection, baseline comparisons and interpretation	25%
Clarity of the management memo and critical discussion	15%
Total	**100%**

Marks reward justified decisions and sound evaluation, not the highest predictive score. A model with poor predictive performance can support an excellent submission if its limitations are identified and the recommendation follows the evidence.

A Instructor preparation checklist

Remove this appendix before distributing the student brief if preferred.

This document is the project specification. The dataset, Excel starter file and executable notebook are separate teaching resources to prepare before launch.

1. **Inspect the actual dataset.** Verify its source and permitted use, target coding, class counts, duplicates, missingness and field definitions. The current specification is based on the supplied dataset summary, not an inspection of the CSV.
2. **Assess feasibility.** Check that each split contains enough successes and failures for meaningful evaluation. Run a leakage-free baseline analysis. If the target is independent of all available inputs, either make non-deployment analysis an explicit learning objective or select a more suitable dataset before teaching.
3. **Approve predictor eligibility.** Establish which financial fields are available before processing. Exclude unverified post-outcome information.
4. **Prepare fixed data partitions.** Reserve 20% for the final test and release the remaining 80% as development data. Use stratification unless a defensible chronological setup is selected. Keep related or duplicate records together where needed.
5. **Prepare the Excel starter file.** Include raw data, a feature sheet and a dictionary template. Specify CSV encoding, separators, date conventions and rules for missing values.
6. **Prepare and test the notebook.** Provide clearly labelled configuration fields and ordered execution instructions. Validate headers, IDs, numeric parsing, binary targets and prohibited columns. Fit imputation, encoding and numerical scaling within training-only pipelines. Handle unseen categories and preserve feature alignment.
7. **Automate outputs.** Include both model families, the prescribed tree depths, threshold sweeps, all cost scenarios, baseline policies, an experiment log, cost plots and review workload. Label undefined metrics explicitly.
8. **Control final testing.** Receive locked settings and Excel transformation recipes, apply them consistently to the held-out inputs, and evaluate the selected fitted pipelines once. Keep final test outcomes unavailable during model selection.
9. **Set teaching logistics.** Specify workload, group size, deadline, access to the execution environment and submission conventions. Pilot the workflow with a learner unfamiliar with Python.